

Examen Parcial de IA

(5 de noviembre de 2020)

Duración: 80 min

1. (4 puntos) Una empresa nos pide un sistema inteligente para acelerar la transmisión de datos entre máquinas. Queremos transmitir X Kbits entre dos máquinas M_I y M_F , para ello hemos de establecer C canales de comunicación que han de atravesar N máquinas ($M_1, M_2, \dots, M_N$) que hacen de puntos intermedios ($C \leq N$), los X Kbits los dividimos a partes iguales por cada canal. Conocemos la velocidad en *kbps* del canal que se puede establecer entre cada par de máquinas (incluidas M_I y M_F), queremos que la velocidad media de un canal sea como mínimo V Kbps y que un canal no atraviese más de P máquinas.

Buscamos la solución que conecte todas las máquinas sin que dos canales pasen por la misma máquina (exceptuando M_I y M_F) tardando lo menos posible en transmitir los X Kbits.

Tras un análisis inicial del problema un compañero de nuestra empresa nos plantea dos estrategias distintas para resolverlo:

- a) Usar el algoritmo de A^* . Definimos el estado como la asignación de máquinas a canales. El estado inicial consiste en asignar las máquinas M_I y M_F a cada canal. Tenemos un operador que asigna una máquina a un canal solo si no se excede el valor P para ese canal. El coste es la velocidad media del canal imaginario que empieza en M_I , pasa por la máquina que estamos asignando y acaba en M_F . Como función heurística usamos la suma de las velocidades medias de todos los canales imaginarios que se obtendrían poniendo cada máquina que queda por conectar como único nodo intermedio entre M_I y M_F .

El algoritmo A^* está pensado para encontrar la solución óptima. Si entendemos que el problema nos pide encontrar la configuración que tarde *el mínimo* tiempo posible, es el algoritmo adecuado para ello. Si entendiéramos que nos pide minimizar en lo posible el tiempo sin encontrar el óptimo, el A^* no justificaria su coste elevado para la resolución del problema, ya que introducimos una complejidad algorítmica innecesaria.

A priori el espacio de búsqueda está bien definido. Se plantea el problema como un espacio de estados en el que partimos de un estado inicial que tiene asignadas solo las máquinas M_I y M_F a cada canal y entre cada estado lo que hacemos es añadir una máquina más a un canal. El estado final sería el haber asignado todas las N máquinas.

El operador va construyendo la solución asignando cada vez una máquina a un canal comprobando que no nos pasemos del número máximo de máquinas por canal, su factor de ramificación es $O(N \times P)$. Pero el operador también debería comprobar que no se repitan máquinas y que se mantenga la restricción de la velocidad media del canal (esta comprobación se puede hacer solo cuando hay al menos una máquina asignada a un canal, y por lo tanto podemos hacerla desde la primera asignación).

El problema principal del coste del operador es que no tiene nada que ver con el tiempo de transmisión en el canal al que se asigna la máquina, ya que lo hace sobre un canal imaginario que conecta la maquina directamente a M_I y M_F . Además este coste, para una máquina, será el mismo independientemente del canal al que la asignemos. El coste debería calcularse como el tiempo que se tarda en transmitir los X bits atravesando todas las máquinas asignadas al canal, que se debe minimizar. Usar la velocidad media como coste no es una buena idea ya que para minimizar el tiempo hemos de maximizarla y A^* solo minimiza.

La función heurística estima exactamente el coste de las máquinas por colocar del mismo modo que el operador que estamos usando para resolver el problema, lleganod a valer cero cuando se han colocado todas. Podríamos decir que el heurístico es admisible, ya que es optimista al suponer que hay una sola máquina en el canal. Pero al igual que en el coste del operador, deberíamos maximizarla y A^* solo minimiza.

El problema es que $f = g + h$ es una función que siempre da un valor constante, ya que el coste de todas las soluciones en este caso será el mismo porque la velocidad media de los canales que usan solo una máquina para conectar M_I y M_F no cambia. El algoritmo, con la g (suma de costes de los operadores) y h (heurístico) propuestas, irá colocando las máquinas en grupos de P en orden creciente de velocidad media (al revés de lo que queremos) hasta colocarlas todas.

Solucionar el problema requeriría usar como coste el tiempo de transmisión, el operador debería contar cuanto aumenta este tiempo al añadir una nueva máquina a un canal y poder estimar cual sería este tiempo para las máquinas restantes. Pero este tiempo restante es bastante difícil de estimar.

- b) Usar un algoritmo de satisfacción de restricciones. El grafo de restricciones tendría como variables las máquinas, los dominios son los canales donde podemos asignarlas. Supondremos que no tenemos en cuenta las máquinas M_I y M_F ya que estarán asignadas a todos los canales. Como restricciones imponemos que un canal no esté asignado a más de P máquinas y que la velocidad media mínima del canal que incluye las máquinas asignadas sea mayor que V .

La aplicación de un algoritmo de satisfacción de restricciones no es adecuada para este problema, ya que no puede optimizar el tiempo de transmisión, por lo que no obtendremos lo que nos pide el problema.

Como representación se plantea elegir las maquinas como variables. Esta elección es correcta, ya que al reducir el dominio de todas las variables a un solo valor sabremos a que canal queda asignada cada una de las máquinas. Además esta representación nos asegura que se asignen todas las máquinas a algun canal, y que cada máquina puede ser asignada a uno y solo uno de los canales que queremos crear, evitando tener que añadir dos restricciones más para controlarlo. La representación contiene N variables con C posibles valores cada una, dando un espacio de búsqueda pequeño y difícil de mejorar.

La restricción sobre el número de máquinas por canal sería una restricción n -aria entre todas las variables, ya que a priori no sabemos que maquinas serán asignadas a cada canal. Al necesitar comprobar un límite superior ($\leq P$) puede ser usada durante la asignación de variables.

La restricción sobre la velocidad media mínima de cada canal seria también una restrucción n -aria entre todas las variables. Esta es imposible de controlar ya que, tal como funcionan los algoritmos de satisfacción de restricciones, empezaremos con una asignación vacía y por lo tanto ningún canal satisfará la restricción. Solo deberíamos comprobarla cuando al menos tengamos una máquina asignada a un canal.

Como se puede ver el planteamiento propuesto es complejo ya que requiere de restricciones n -arias englobando todas las variables. Pero no hay otros planteamientos más simples que funcionen. El planteamiento de hacer que las variables sean los canales y los dominios sean los identificadores de máquina seria incorrecto, ya que un canal puede tener más de una máquina asignada. El planteamiento de tener P variables por cada canal ($C_i1..C_iP$, representando las P máquinas que como máximo se puede asignar al canal), donde el dominio de cada variable son todos los posibles identificador de máquinas (añadiendo el valor vacío para permitir canales con menos de P máquinas asignadas tendría la ventaja de que la restricción de máximo P máquinas por canal queda asegurada por la representación y que podemos asegurar que no se repitan máquinas añadiendo un grafo completo de restricciones binarias de desigualdad entre todas las variables, pero aun así tendríamos que tener una restricción n -aria entre todas las variables de un mismo canal para comprobar la velocidad media, y otra restricción n -aria abarcando todas las variables del grafo para asegurar que toda maquina está asignada en algun canal. Teniendo en cuenta que en ese caso tenemos $C \times P$ variables con un dominio de $N + 1$ valores, y que $C \leq N$, esta otra posibilidad seria más compleja que la propuesta en el enunciado.

Comenta cada una de las soluciones que se proponen, analizando si la técnica escogida es adecuada para este problema, si cada uno de los elementos de la solución son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

2. (6 puntos) Los dueños del hotel “MiniPreu” quieren reducir su presupuesto de limpieza para maximizar sus ganancias. Para hacerlo han decidido tener tres tipos diferentes de precios para sus habitaciones. El precio A incluye una limpieza diaria de la habitación y tiene un precio de pa euros por día, el precio B incluye una limpieza de la habitación en días alternos y tiene un precio de pb euros por día y el precio C solo incluye una limpieza el primer y último día y tiene un precio de pc por día. El mínimo número de días que se pueden reservar para una habitación es de dos, y asumiremos que la limpieza de las habitaciones de precio B esta organizada de manera que la última limpieza se hace el día en el que el cliente deja su habitación. Tenemos un total de R habitaciones en el hotel.

Limpiar una habitación de precio A cuesta ca euros por día de limpieza, una habitación de precio B cuesta cb por día de limpieza y una habitación de precio C cuesta ca más el número de días que la habitación ha sido reservada multiplicado por el coste cc . Las habitaciones que están vacías no se limpian.

El departamento de sanidad impone como restricción que toda habitación (ocupada o no) ha de limpiarse al menos 10 veces al mes. Esto fuerza a que ninguna habitación pueda quedar sin ocupar muchos días.

El hotel recibe peticiones de reserva para un mes completo, cada reserva incluye el precio que el cliente quiere, el día en el que la reserva comienza y el número de días a reservar. Asumiremos que todas las peticiones comienzan y finalizan dentro del mes. El objetivo es decidir qué reservas aceptar, de manera que los gastos de limpieza se minimicen y las ganancias se maximicen.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (solución inicial, operadores, función heurística, ...). El objetivo es comentar la solución que se propone, analizando si la técnica escogida es adecuada para este problema, si cada uno de los elementos de la solución son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

- a) Queremos usar Hill-climbing, la solución inicial es la solución vacía. Como operadores de búsqueda usamos **añadir-reserva** que añade una reserva de una habitación, **cambiar-reserva** que cambia la reserva de una habitación por otra reserva que no esté ya en la solución. La función heurística es la suma de todos los costes de limpieza de las habitaciones multiplicado por las ganancias obtenidas por las reservas.

Plantear como solución un Hill Climbing para este problema es adecuado a priori, ya que nos piden encontrar una solución maximizando o minimizando una serie de criterios sin necesidad de obtener el óptimo.

La solución inicial planteada no es solución (las habitaciones se han de limpiar al menos 10 veces al mes y en una solución vacía ninguna habitación es limpiada). El coste de generación es $O(1)$ pero su calidad es mala ya que se está muy lejos de cualquier solución, dejando todo el trabajo de construir una primera solución al algoritmo de búsqueda. Podríamos empezar con esta solución si se asegura que el algoritmo de búsqueda acabará en una solución que cumpla las restricciones.

Los operadores de búsqueda permiten cubrir todo el espacio de soluciones ya que pueden generar cualquier configuración (solución o no) para el problema. De hecho el operador **añadir-reserva** ya nos asegura el poder construir potencialmente cualquier solución a partir de la solución vacía, pero al ser un Hill-Climbing que siempre escoge el mejor sucesor local, el operador **cambiar-reserva** permitirá ajustar la calidad de la solución sobretudo

en las últimas iteraciones de la búsqueda. Sin embargo estos operadores pueden pasar de una solución a una no-solución, ya que no mantienen la restricción de que cada habitación se ha de limpiar más de 10 veces al mes, y no comprueban si las reservas se solapan con las que ya están en la solución. Ambos operadores deberían comprobar siempre si es posible añadir una reserva en la habitación específica (no hay otra reserva ya durante alguno de los días de la reserva nueva). La restricción del mínimo de 10 limpiezas por habitación no se cumple al inicio de la búsqueda, por lo que es mejor tratarla directamente en el heurístico. El factor de ramificación de los operadores en el caso peor es el producto de las reservas y el número de habitaciones (en el caso de **añadir-reserva**) y el cuadrado de las reservas en el otro (en el caso de **cambiar-reserva**).

La función heurística asigna el mismo valor a soluciones que tienen un bajo coste de limpieza y gran beneficio que a soluciones con gran coste de limpieza y bajo beneficio, por lo que no puede distinguir las buenas soluciones de las malas. Minimizar este heurístico no es una buena idea ya que a menores valores menor es el beneficio. Además este heurístico debería incluir una penalización de las no soluciones que permita distinguir entre una no-solución mala y una mejor, de forma que pueda guiar la búsqueda por el espacio de no-soluciones dirigiéndola hacia el espacio de soluciones.

- b) Queremos usar Hill-climbing, la solución inicial se obtiene de la siguiente manera: para una habitación elegimos una de las reservas que tenga la fecha más temprana, la siguiente reserva será la que tenga la fecha más temprana que empiece después del final de la última reserva asignada a la habitación, repetimos este procedimiento hasta que no podamos asignar más reservas a la habitación. Hacemos esto para todas las habitaciones. Como operadores de búsqueda usamos **cambiar-reserva** que cambia una reserva de una habitación por otra reserva que no este en la solución. La función heurística es la suma de todos los costes de limpieza de las habitaciones menos las ganancias obtenidas por las reservas.

Como se ha dicho en el apartado anterior, a priori Hill Climbing es adecuado para este tipo de problema.

La solución inicial puede ser no-solución, ya que no hay garantía de que la solución inicial cumpla la restricción de limpiar cada habitación más de 10 veces al mes (si hay pocas reservas podría ocurrir que todas se concentren en las primeras habitaciones, dejando las últimas habitaciones con pocas o ninguna reserva; pero incluso si hay más reservas de las que se pueden colocar en las habitaciones, si en una habitación se concentran muchas reservas de tipo C de más de 3 días, esa habitación podría no limpiarse 10 veces al mes. No hay tampoco garantía sobre la calidad de la solución, ya que esta solución inicial es un greedy que solo se centra en llenar las habitaciones con el máximo número de reservas, pero no tiene en cuenta los tipos de precio de las habitaciones o los costes de limpieza y por lo tanto puede estar lejos de un máximo local (con respecto al beneficio neto obtenido). El coste de generación de la solución inicial, asumiendo que ordenamos primero las reservas por fecha de inicio, es $O(N + \log N) + N \times R$, siendo N el número de reservas y R el número de habitaciones, ya que una vez ordenadas las reservas por habitación, para cada habitación hemos de recorrer la lista de reservas que quedan para ir las seleccionando en orden de fecha.

El operador no puede generar todas las posibles soluciones ya que el número de reservas de la solución inicial no se puede cambiar. El operador no comprueba si la reserva cabe en el calendario de la habitación o que la habitación sea limpiada más de 10 veces al mes. En el caso extremo de que la solución inicial haya podido colocar todas las reservas en habitaciones, el operador no tendría reservas fuera de la solución para intercambiar. Es por todo ello que sería necesario añadir al menos un operador para **mover** una reserva de una habitación a otra. El factor de ramificación del operador en el peor caso es $O(N^2)$, siendo N el número de reservas.

La función heurística propuesta suma los costes y resta los beneficios. Minimizando la

función heurística propuesta podemos obtener el tipo de soluciones que queremos. Sin embargo el heurístico no es correcto ya que debería incluir una penalización de las no soluciones.

- c) Se plantea utilizar algoritmos genéticos. Asignamos a cada reserva un número de 0 a R , la concatenación de estos números para todas las reservas en binario es la codificación de una solución. El número 0 significa que la reserva no está asignada a ninguna habitación, otro número representa el número de habitación asignada a la reserva. Para generar la población inicial obtenemos una solución asignando aleatoriamente una reserva a cada habitación (solo R reservas tienen un número diferente de 0). Como operadores genéticos usamos los operadores habituales de cruce y mutación. La función heurística es la suma para todas las reservas en la solución del cociente entre las ganancias obtenidas por la reserva y los costes de limpieza de la reserva.

Plantear como solución un algoritmo genético para este problema es adecuado a priori, ya que nos piden encontrar una solución maximizando o minimizando una serie de criterios sin necesidad de obtener el óptimo.

La codificación de las soluciones es una representación correcta del problema, es la habitación que ha de asignarse a cada reserva, y requiere solo $N \times \log R$ bits, siendo N el número de reservas y R el número de habitaciones. La representación escogida permite representar cualquier solución del problema pero también muchas no-soluciones: identificadores de habitación que no existen, habitaciones con reservas solapadas, habitaciones sin un mínimo de 10 limpiezas.

La estrategia para generar la población obtiene diferentes soluciones iniciales, esto es correcto. Pero su variabilidad es limitada, ya que el número de reservas diferentes en la población inicial depende del tamaño de la población P (si el tamaño de la población es pequeño, solo unas pocas reservas serán utilizadas para obtener nuevos individuos). Otro problema es que las soluciones iniciales pueden no cumplir la restricción de limpiar las habitaciones más de 10 veces al mes ya que cada habitación tiene una sola reserva, y tanto si son reservas A o B de pocos días como si es una reserva C de cualquier duración no cumplirán las limpiezas mínimas. El coste de generación es $O(P \times R)$, siendo P el tamaño escogido de la población de soluciones iniciales y R el número de habitaciones.

Los operadores habituales pueden generar soluciones y todos los tipos de no-solución que permite la representación: habitaciones asignadas a reservas que se solapan en el tiempo, identificadores de habitación que no existen, habitaciones que no cumplen la restricción de limpiar más de 10 veces al mes, etc. En este caso el operador de mutación genera soluciones muy próximas a las originales (solo cambia el identificador de habitación de una reserva en un bit) y es el operador de cruce el que genera sucesores diferentes a los padres (pero no muy lejos de ellos).

La función heurística aparentemente generará las soluciones que queremos, ya que cuanto mayor sea el cociente entre ganancia y gasto mejor es una reserva, pero podemos obtener el mismo cociente con diferentes valores y la ganancia neta obtenida no será la misma, por lo que no nos distingue bien entre soluciones diferentes. Además este heurístico debería incluir una penalización de las no soluciones.